

Cours 5

Programmation orientée objet

Jérémy Kairis

Les espaces de noms

Les espaces de noms

Définition

Le Framework .Net est structuré en espaces de noms. On peut les comparer à un système de fichiers.

Chaque répertoire serait un conteneur qui peut contenir d'autres répertoires ou des fichiers et où chaque fichier serait un type de donnée.

Au contraire du système de fichiers, cette structure est une structure logique et non physique.

Les espaces de noms

namespace

L'espace de noms dans lequel va se trouver un type, c'est la valeur définie après le mot-clé “*namespace*”.

Lorsqu'un nouveau type est défini, il porte un nom complet
« *Namespace.NomDuType* ».

```
using System;

namespace Cours
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Hi world");

            System.Console.WriteLine("...");
        }
    }
}
```

Les espaces de noms

using

Le mot-clé « ***using*** » importe les types d'un espace de noms défini.

Ajouter « ***using System;*** » donne accès au type « ***Console*** » sans devoir spécifier :« ***System.Console*** ».

Les espaces de noms

Exercise

Créer un namespace ***Bibliothèque*** avec une classe ***Livre*** :

- Attributs : ***Titre, Auteur, AnneePublication***
- Constructeur + méthode ***AfficherInfo()***

Créer un sous-namespace ***Bibliothèque.Utilisateurs*** avec une classe ***Etudiant*** :

- Attributs : ***Nom, Prenom, Livres***
- Constructeur + méthode ***AfficherInfo()***

Afficher les informations d'un étudiant et de ces livres

L'encapsulation

L'encapsulation

Définition

L'encapsulation est un principe fondamental au sein de l'orienté objet.

Elle permet de protéger et de contrôler l'accès aux types afin de garantir l'intégrité des informations qu'ils contiennent.

Pour cela, nous devons donner à nos types et à leurs membres, un modificateur d'accès qui limitera l'accès à ceux-ci.

Voiture

-Moteur moteur

+DemarrerVoiture() : void

+ArreterVoiture() : void

Moteur

+int Cylindres

+double PuissanceEnCv

+Demarrer() : void

+Arreter() : void

L'encapsulation

Modificateurs d'accès

Les principaux modificateurs d'accès sont :

- ***public*** : accessible partout (+)
- ***protected*** : accessible dans la classe même et ces enfants (héritage)
- ***private*** : accessible dans la classe même (-)
- ***internal*** : accessible partout au sein de l'assembly (du projet)


```
public class Moteur
{
    private int cylindres;
    private double puissanceEnCv;

    public Moteur(int cylindres, double puissanceEnCv)
    {
        this.cylindres = cylindres;
        this.puissanceEnCv = puissanceEnCv;
    }

    public int Cylindres => cylindres;
    public double PuissanceEnCv => puissanceEnCv;

    public void Demarrer() => Console.WriteLine("Le moteur démarre !");
    public void Arrêter() => Console.WriteLine("Le moteur s'arrête.");
}
```

```
public class Voiture
{
    private Moteur moteur;

    public Voiture(Moteur moteur)
    {
        this.moteur = moteur;
    }
    public void DemarrerVoiture()
    {
        Console.WriteLine("La voiture démarre !");
        moteur.Demarrer();
    }
    public void ArreterVoiture()
    {
        Console.WriteLine("La voiture s'arrête.");
        moteur.Arreter();
    }
}
```

```
var moteurV6 = new Moteur(6, 300.0);  
var maVoiture = new Voiture(moteurV6);
```

```
maVoiture.DemarrerVoiture();  
maVoiture.ArreterVoiture();
```


L'encapsulation

Exercices

- Créer un conducteur pour conduire la voiture
- Ajouter le moteur et des roues à la voiture

Les propriétés

Les propriétés

Introduction

Les propriétés permettent de contrôler l'accès et le contenu de nos variables de classe grâce aux accesseurs (***get***) et aux mutateurs (***set***) en assurant leurs intégrités.

Elles fournissent une interface pour lire, écrire ou calculer des valeurs d'une classe sans exposer directement les champs privés.

```
public class Personne
{
    private int age;
    public int Age
    {
        get { return age; }
        set { age = value; }
    }
}
```

// ou

```
public int Age { get; set; }
}
```

```
public class Personne
{
    private int age;

    public int GetAge()
    {
        return age;
    }

    public void SetAge(int value)
    {
        age = value;
    }
}
```


Les propriétés

Accessibilité

L'accessibilité des propriétés peut être modifiée comme pour empêcher la modification de la propriété après son initialisation.

Pour se faire, il est possible de ne définir que le ***get***, utiliser le ***init***, ou le ***private set*** selon les besoins.

```
public class Personne
{
    public bool EstEudiant => true;
    public int Age { get; }
    public string Nom { get; init; } = string.Empty;
    public DateTime DateDeNaissance { get; private set; }
}
```

Les propriétés

Vérifications

Comme des méthodes, les propriétés permettent d'ajouter des vérifications sur les données avant de les lire ou de les écrire.


```
class Personne
{
    private int age;
    public int Age
    {
        get { return age; }
        set
        {
            if (value >= 0 && value <= 120)
            {
                age = value;
            }
            else
            {
                throw new InvalidDataException();
            }
        }
    }
}
```

Les propriétés

Transformation

Comme des méthodes, les propriétés permettent de transformer les données avant de les lire ou de les écrire.


```
public class Personne
{
    public required string Nom { get; set; }

    public required string Prénom { get; set; }

    public string Appellation => $"{Prénom} {Nom}";
}
```

Les propriétés

required

Le mot-clé ***required*** assure qu'une valeur sera assignée à la propriété lors de la création de l'objet.

Les propriétés

Utilisation

L'utilisation des propriétés est équivalente à celle des variables, le ***set*** est appelé lors de l'assignation de la valeur et le ***get*** lors de la récupération de la valeur.


```
Personne p = new();
```

```
p.Age = 33;
```

```
var age = p.Age;
```

```
public class Personne  
{  
    public int Age { get; set; }  
}
```

Les propriétés

Le *null* conditionnel ?.

Il existe un variant à l'opérateur d'accès aux membres “.” .

Cette variante est l'opérateur null conditionnel “?.”, ce dernier vérifie que la variable est non “***null***” avant d'accéder au membre souhaité.

Si la variable est “***null***”, il ne fera rien.

```
Personne? p = null;
```

```
// Si `p` est null, `ville` sera également null sans générer d'exception.
```

```
string? ville = p?.Adresse?.Ville;
```

```
Console.WriteLine(ville);
```

```
public class Personne
```

```
{
```

```
    public string? Nom { get; set; }
```

```
    public Adresse? Adresse { get; set; }
```

```
}
```

```
public class Adresse
```

```
{
```

```
    public string? Ville { get; set; }
```

```
}
```


Exercice

Banque

Créez un nouveau projet sur base de <https://github.com/jeremykairis/bank-2025>.

1. Créer une classe « Person » implémentant :

- Les propriétés publiques
 - string FirstName
 - string LastName
 - DateTime BirthDate

Exercice

Banque

2. Créer une classe « CurrentAccount » qui permet la gestion d'un compte courant implémentant :

- Les propriétés publiques
 - string Number
 - double Balance (lecture seule)
 - double CreditLine
 - Person Owner
- Les méthodes publiques
 - void Withdraw(double amount)
 - void Deposit(double amount)

Exercice

Banque

3. Créer une classe « Bank » pour gérer les comptes de la banque implémentant :

- Les propriétés
 - Dictionary<string, CurrentAccount> Accounts (lecture seule)
 - string Name
- Les méthodes :
 - void AddAccount(CurrentAccount account)
 - void DeleteAccount(string number)

Exercice

Banque

4. Ajouter une méthode qui retourne le solde d'un compte courant.
5. Permettre à la banque de donner la somme de tous les comptes d'une personne.